



```
In [2]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error
import tensorflow as tf
```

```
In [12]: data = np.genfromtxt('/content/Fertility_Diagnosis.txt', delimiter=',')

X = data[:, :-1]
y = data[:, -1].reshape(-1, 1)

split = int(0.75 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

```
In [13]: input_size = X_train.shape[1]
output_size = 1

training_errors = []
testing_errors = []

error_tolerance = 0.05
alpha = 0.001
```

Hidden Layer Loop + Feedforward and Backpropagation and Evaluate on Training and Testing data

```
In [14]: def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

for hidden_neurons in range(1, 10):
    np.random.seed(42)

    synapse_0 = 2 * np.random.random((input_size, hidden_neurons)) - 1
    synapse_1 = 2 * np.random.random((hidden_neurons, output_size)) - 1

    for i in range(1, 1000001):
        layer_0 = X_train
        layer_1 = sigmoid(np.dot(layer_0, synapse_0))
        layer_2 = sigmoid(np.dot(layer_1, synapse_1))

        layer_2_error = y_train - layer_2
        mse = np.mean(np.square(layer_2_error))

        if i % 100000 == 0 or mse < error_tolerance:
            print(f"Hidden Neurons: {hidden_neurons}, Iter: {i}, Training MSE:
```

```

if mse < error_tolerance:
    print(f"Converged at iteration {i} for {hidden_neurons} neurons\n"
          break

layer_2_delta = layer_2_error * sigmoid_derivative(layer_2)
layer_1_error = layer_2_delta.dot(synapse_1.T)
layer_1_delta = layer_1_error * sigmoid_derivative(layer_1)

synapse_1 += layer_1.T.dot(layer_2_delta) * alpha
synapse_0 += layer_0.T.dot(layer_1_delta) * alpha

layer_1_train = sigmoid(np.dot(X_train, synapse_0))
layer_2_train = sigmoid(np.dot(layer_1_train, synapse_1))
train_mse = mean_squared_error(y_train, layer_2_train)
training_errors.append(train_mse)

layer_1_test = sigmoid(np.dot(X_test, synapse_0))
layer_2_test = sigmoid(np.dot(layer_1_test, synapse_1))
test_mse = mean_squared_error(y_test, layer_2_test)
testing_errors.append(test_mse)

print(f"Final -> Neurons: {hidden_neurons} | Train MSE: {train_mse:.6f} |

```

Hidden Neurons: 1, Iter: 100000, Training MSE: 0.086423
Hidden Neurons: 1, Iter: 200000, Training MSE: 0.083058
Hidden Neurons: 1, Iter: 300000, Training MSE: 0.081305
Hidden Neurons: 1, Iter: 400000, Training MSE: 0.080090
Hidden Neurons: 1, Iter: 500000, Training MSE: 0.079154
Hidden Neurons: 1, Iter: 600000, Training MSE: 0.078402
Hidden Neurons: 1, Iter: 700000, Training MSE: 0.077784
Hidden Neurons: 1, Iter: 800000, Training MSE: 0.077269
Hidden Neurons: 1, Iter: 900000, Training MSE: 0.076833
Hidden Neurons: 1, Iter: 1000000, Training MSE: 0.076461
Final -> Neurons: 1 | Train MSE: 0.076461 | Test MSE: 0.105876

Hidden Neurons: 2, Iter: 100000, Training MSE: 0.083758
Hidden Neurons: 2, Iter: 200000, Training MSE: 0.078997
Hidden Neurons: 2, Iter: 300000, Training MSE: 0.076333
Hidden Neurons: 2, Iter: 400000, Training MSE: 0.074502
Hidden Neurons: 2, Iter: 500000, Training MSE: 0.073137
Hidden Neurons: 2, Iter: 600000, Training MSE: 0.072074
Hidden Neurons: 2, Iter: 700000, Training MSE: 0.071232
Hidden Neurons: 2, Iter: 800000, Training MSE: 0.070553
Hidden Neurons: 2, Iter: 900000, Training MSE: 0.069996
Hidden Neurons: 2, Iter: 1000000, Training MSE: 0.069533
Final -> Neurons: 2 | Train MSE: 0.069533 | Test MSE: 0.087332

Hidden Neurons: 3, Iter: 100000, Training MSE: 0.060694
Hidden Neurons: 3, Iter: 186945, Training MSE: 0.050000
Converged at iteration 186945 for 3 neurons

Final -> Neurons: 3 | Train MSE: 0.050000 | Test MSE: 0.127991

Hidden Neurons: 4, Iter: 100000, Training MSE: 0.050415
Hidden Neurons: 4, Iter: 101862, Training MSE: 0.050000
Converged at iteration 101862 for 4 neurons

Final -> Neurons: 4 | Train MSE: 0.050000 | Test MSE: 0.152829

Hidden Neurons: 5, Iter: 100000, Training MSE: 0.050273
Hidden Neurons: 5, Iter: 101004, Training MSE: 0.050000
Converged at iteration 101004 for 5 neurons

Final -> Neurons: 5 | Train MSE: 0.050000 | Test MSE: 0.102242

Hidden Neurons: 6, Iter: 100000, Training MSE: 0.063655
Hidden Neurons: 6, Iter: 200000, Training MSE: 0.055934
Hidden Neurons: 6, Iter: 300000, Training MSE: 0.054370
Hidden Neurons: 6, Iter: 400000, Training MSE: 0.053902
Hidden Neurons: 6, Iter: 500000, Training MSE: 0.051518
Hidden Neurons: 6, Iter: 500742, Training MSE: 0.049999
Converged at iteration 500742 for 6 neurons

Final -> Neurons: 6 | Train MSE: 0.049999 | Test MSE: 0.094986

Hidden Neurons: 7, Iter: 75662, Training MSE: 0.050000
Converged at iteration 75662 for 7 neurons

Final -> Neurons: 7 | Train MSE: 0.050000 | Test MSE: 0.105181

Hidden Neurons: 8, Iter: 82251, Training MSE: 0.050000
Converged at iteration 82251 for 8 neurons

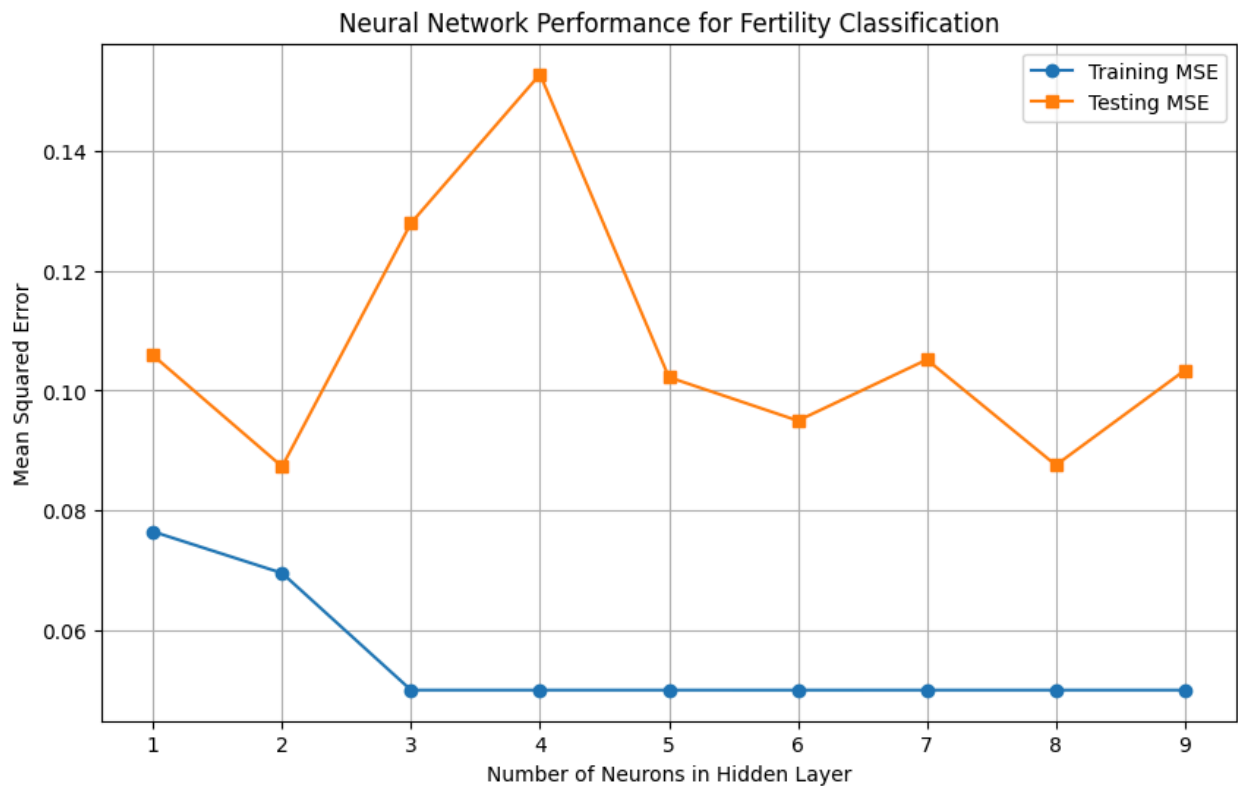
Final -> Neurons: 8 | Train MSE: 0.050000 | Test MSE: 0.087563

Hidden Neurons: 9, Iter: 96465, Training MSE: 0.050000
Converged at iteration 96465 for 9 neurons

Final -> Neurons: 9 | Train MSE: 0.050000 | Test MSE: 0.103399

Plotting the results

```
In [15]: plt.figure(figsize=(10, 6))
plt.plot(range(1, 10), training_errors, marker='o', label='Training MSE')
plt.plot(range(1, 10), testing_errors, marker='s', label='Testing MSE')
plt.xlabel('Number of Neurons in Hidden Layer')
plt.ylabel('Mean Squared Error')
plt.title('Neural Network Performance for Fertility Classification')
plt.legend()
plt.grid(True)
plt.show()
```



Report:

1. What happens if all the weights are initialized to zero (zero initialization)?

If all weights are initialized to zero, all neurons learn the same features during training. This is called the symmetry problem. Every neuron receives identical gradients; No differentiation between neurons; Network behaves like a single neuron, even if multiple exist; Learning becomes ineffective

2. Difference between epoch, batch, and iteration

Epoch: One complete pass through the entire training dataset

Batch: A subset of the training data used for one forward/backward pass.

Iteration: One update of weights using a single batch

3. What is the key difference between stochastic gradient descent and batch gradient descent?

The key difference between stochastic gradient descent and batch gradient descent is:

Stochastic Gradient Descent (SGD) updates the weights using one training sample at a time. It is faster and can escape local minima due to its noisy updates, but the path to convergence is less stable.

Batch Gradient Descent updates the weights using the entire training dataset at once. It provides stable and accurate updates but is slower and computationally expensive for large datasets.

4. What problems does a completely random initialization of a neural network lead to?

Can lead to vanishing gradients (very small updates); Can cause exploding gradients (very large updates) Slow convergence; Poor performance if weights are too large or too small

5.ame any ten hyperparameters in a multi-layer neural network.

Ten hyperparameters in a multi-layer neural network are as follows:

- i) Number of hidden layers
- ii) Number of neurons per layer
- iii) Batch size
- iv) Number of epochs
- v) Weight initialization method
- vi) Activation function
- vii) Dropout rate
- viii) Optimizer like SGD, Adam
- ix) Regularization parameters like L1, L2
- x) Learning rate